



Master DSA from Scratch: The A2Z Roadmap (No Shortcuts)

Prepared by: Karsh Tandon

This document breaks down Data Structures & Algorithms into **6 Strategic Levels**. Don't jump around; follow the order to build a solid foundation.

NOTE: For Tutorials you can use any of the resources you want, I have added multiple.

The key part which separates you from others is practicing in the right way, knowing/identifying DSA patterns, ability to handle ambiguity/edge cases and solving problems in a high pressure environment.

Level 1: Programming Foundation (Language & Logic)

Goal: Pick **one** language (C++ or Java recommended) and master its standard library. **Focus:** Syntax, Data Types, Loops, Functions, and **STL** (C++) or **Collections** (Java).

Resources:

- [C++ basics /Java basics](#)
- **C++ STL:**
 - [Striver's C++ STL One Shot \(YouTube\)](#)
 - [GeeksForGeeks STL Guide](#)
- **Java Collections:**
 - [Kunal Kushwaha's Java Playlist](#)

Level 2: The Basics (Complexity & Linear DS)

Goal: Understand how to measure code efficiency and learn basic storage. **Topics:** Time & Space Complexity (Big O), Arrays, Linked Lists, Stacks, Queues.

Resources:

- **Complexity:** [Abdul Bari's Algorithms Playlist](#) (Watch first 2-3 videos)
- **Visualization:** [VisuAlgo.net](#) (Visualize how Linked Lists/Stacks work)
- **Practice Sheet:** [Striver's A2Z Sheet - Basics Step](#)

- Cheatsheet: [Big-O Cheatsheet](#) (Keep this bookmarked)
-

Level 3: The Interview Core (Algorithms & Patterns)

Goal: The 70% zone. Don't just solve; learn **Patterns**. **Topics:** Sorting, Binary Search, Hashing, Two Pointers, Sliding Window.

Resources:

- [C++_dsa](#) / [Java_dsa](#)
 - Sliding Window: [Aditya Verma's Sliding Window Playlist](#)
 - Binary Search: [Striver's Binary Search Series](#)
 - Practice Resource: [DSA patterns](#)
-

Level 4: Trees, Heaps, Recursion

Goal: Master non-linear data structures. **Topics:** Recursion, Trees (BST), Heaps, Backtracking, Greedy. **Strategy:** Use the "Recursion Tree" method to trace calls.

Resources:

- Recursion: [Peppcoding Recursion Playlist](#) (Hindi - Excellent for intuition)
- Trees: [TakeUForward Binary Trees](#)
- Heaps: [Aditya Verma Heap Playlist](#)

Checkpoint: Before Level 5, ensure you can solve Medium-level problems on LeetCode for these topics.

Resources: [DSA patterns](#) [Made by me, has links for different topics as well as links for dsa patterns for all topics]

Level 5: DP & Graphs

Goal: The "Filter" rounds. If you master these, you are in the top 10%.

Topics: 1D/2D DP, BFS/DFS, Dijkstra, Topological Sort, DSU.

Resources:

- **Dynamic Programming:**
 - YT: [Aditya Verma's DP Playlist](#)
 - [Striver](#)
 - Read: [TopCoder DP Tutorial](#)
 - Leetcode: <https://leetcode.com/discuss/post/1000929/solved-all-dynamic-programming-dp-problem-8m82/> (Practice few questions from each dp topic)
 - **Graphs:**
 - [Striver's Graph Series](#)
 - [William Fiset Graph Theory](#) (Best visual explanations)
 - Practice: [DSA patterns](#)
-

Level 6: Advanced (For Top Tier) [DO at the end if time permits]

Goal: For HFTs (Tower Research, Graviton) or Google/CodeNation hard rounds.

Topics: Tries, Segment Trees, Fenwick Trees, Bit Manipulation.

Resources:

- **CP-Algorithms:** [CP-Algorithms.com](https://cp-algorithms.com/) (The Bible for advanced algos)
 - **Standard Problems:** [CSES Problem Set](#) (Solve the first 50-100)
-

The Practice Phase (Most Important)

1. **Pattern Recognition:** Don't memorize questions, identify patterns.
2. **Contests:** Give **LeetCode Weekly Contests** (simulates pressure) or Give **virtual Contests [MUST]**
3. **Topic-less Practice:** Use LeetCode's "Pick One" to test random recall and practice without knowing the **question topic**.

Resources: [DSA patterns](#) [Cover few questions from all the patterns in the links provided, once you do few questions, then practice more questions on that. This makes sure you atleast know of most dsa patterns.]

Tips:

1. Build problem solving skills and pattern identification skills so that you can solve similar problems in future.

2. Brainstorm atleast for 20-30mins from yourself before looking at the solution
3. Try to code within a given time frame to simulate interview coding.[USE A TIMER]
4. **It takes months and months of practice to become good at DS, so KEEP PRACTICING IF YOU WANT A HIGH PACKAGE.**
5. Always ask edge cases, constraints from the interviewer and DO NOT jump straight to the solution. Explain time and space complexity for every question.

Interview Specifics

- **Mock Interviews:**
 - [Pramp](#) (Free peer-to-peer)
 - [Interviewing.io](#) (Anonymous with engineers)
- **Behavioral:** [Tech Interview Handbook - Behavioral](#)
- **Past Interview Experiences:** Read [LeetCode Discuss](#) [MY FAV WEBSITE, Also consists of compensations etc for different roles], [Medium.com](#) blogs and [GeeksForGeeks Archives](#). [Pick any 2]

Other resources:

<https://neetcode.io/roadmap>

All the best! Follow for more updates.